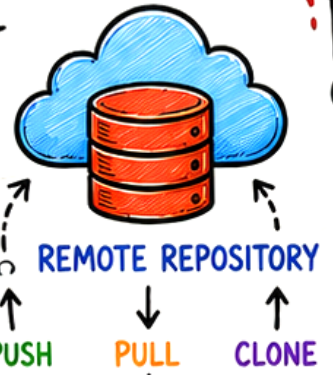


TRACK CHANGES, CODE TOGETHER!

GIT

COMPLETE NOTES

BEGINNER TO ADVANCED



```
$ git init
$ git add .
$ git commit -m
$ git push
```

WHAT IS GIT?

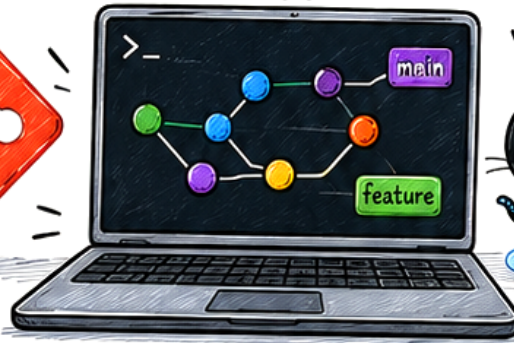
Git is a **distributed** version control system that helps you track changes, collaborate and manage code efficiently.



WHY GIT?

- ✓ Version Control
- ✓ Collaboration
- ✓ Backup & Safety
- ✓ Branching
- ✓ History Tracking
- ✓ Open Source

THE GIT WORKFLOW



ESSENTIAL GIT COMMANDS

```
$ git init # Initialize repo
$ git add . # Stage changes
$ git commit -m "msg" # Commit changes
$ git status # Check status
$ git log # View history
$ git branch # List branches
$ git checkout <branch> # Switch branch
$ git merge <branch> # Merge changes
$ git push # Push to remote
$ git pull # Pull from remote
```

GIT HELPS YOU...



SAFE CODE BACKUP



TEAM COLLABORATION



EASY BRANCHING



TRACK HISTORY



FASTER DEVELOPMENT



RELIABLE & SECURE

COVERS EVERYTHING YOU NEED



GET STARTED

- ✓ Git Basics
- ✓ Installation
- ✓ Configuration
- ✓ Repositories
- ✓ Commits



WORKING WITH CODE

- ✓ Staging Area
- ✓ Adding Files
- ✓ Committing
- ✓ Ignoring Files
- ✓ Git Status



BRANCHING & MERGING

- ✓ Branches
- ✓ Checkout
- ✓ Merge
- ✓ Rebase
- ✓ Cherry Pick



REMOTE OPERATIONS

- ✓ Remote Repos
- ✓ Push
- ✓ Pull
- ✓ Fetch
- ✓ Clone



ADVANCED TOPICS

- ✓ Rebase & Merge
- ✓ Stash
- ✓ Tags
- ✓ Submodules
- ✓ Worktrees



BEST PRACTICES

- ✓ Git Flow
- ✓ Collaboration
- ✓ Code Review
- ✓ Clean History
- ✓ Tips & Tricks



★ LEARN **GIT**. MASTER CONTROL. CODE **BETTER** TOGETHER! ★



Introduction to Git



1

1 What is Git?

Git is a **Distributed Version Control System (DVCS)** used to track changes in source code during software development.



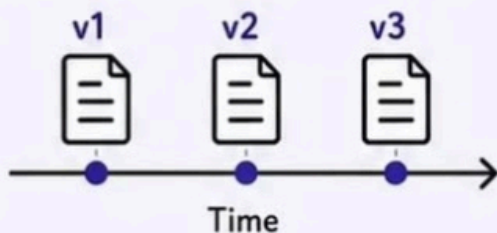
Why Git?



- ✓ Track every change
- ✓ Work with team easily
- ✓ Go back to any version
- ✓ Create branches & features
- ✓ Safe, fast & reliable

2 What is Version Control?

Version Control is a system that records changes to files over time so you can recall specific versions later.

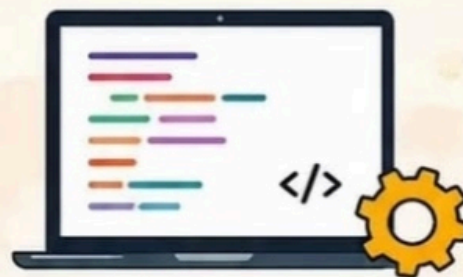


3 Git vs GitHub

Git	VS	GitHub
Tool (Version Control System)		Platform (Cloud Service)
Works on your local machine		Hosts Git repositories
Command line based		Web based interface
Manages code history		Collaboration & sharing

4 Why Git is Important?

- ★ Every developer & tester should know Git.
- ★ Industry standard skill.
- ★ Essential for collaboration.
- ★ Helps in backup, tracking, and managing code efficiently.



5 Key Benefits of Git

Collaboration Multiple developers can work together without overwriting each other's code.	Branching Create branches to work on new features without affecting main code.	Safety Full history of changes is stored. You can recover anytime.	Backup Your code is always safe in remote repositories.	Speed Git is lightweight and performs operations very fast.	Open Source Free, open-source & used by millions worldwide.
--	--	--	---	---	---



Remember



Git helps you track changes, collaborate, and manage code like a pro developer!





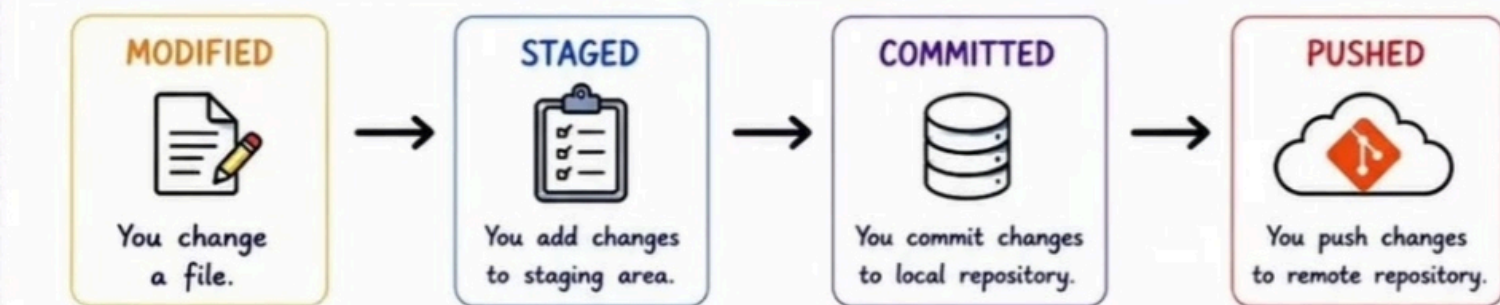
GIT ARCHITECTURE

2

Git works like a **4-layer system** where your changes move step by step from your computer to the **remote server**.



GIT LIFECYCLE



EASY ANALOGY

- Working Directory → Your Desk
- Staging Area → Your Clipboard
- Local Repository → Your Locker
- Remote Repository → Your Cloud Locker (Team Storage)

IMPORTANT COMMANDS IN FLOW

- 1** `git status` → Check current state
- 2** `git add <file>` → Move to staging area
- 3** `git commit -m "message"` → Save locally
- 4** `git push origin <branch>` → Push to remote

TIP

Always think:

Modify



Add



Commit



Push

This is the **heart** of Git!



Remember:

Commit locally as often as you can.
Push when your changes are ready to **share**.

More commits = **better history**
Smaller commits = **easier to manage**



INSTALLATION & CONFIGURATION

3

Set up Git on your system and configure your identity.



1 Install Git



Download and install Git from the official website.



Official Site:
<https://git-scm.com/downloads>

Choose your operating system:



Windows



macOS



Linux



Follow the installation wizard for your operating system.

2 Check Git Installation

Verify Git is installed successfully.

```
$ git --version
git version 2.44.0.windows.1
```



Tip:

If command is **not found**, restart your terminal.



3 Configure Username & Email

Set your global username and email.

```
$ git config --global user.name "Your Name"
# sets the username

$ git config --global user.email "youremail@example.com"
# sets the email
```

These details will be attached to all your commits.



4 View Current Configuration



Check your current Git configuration.

```
$ git config --list
# shows all global configuration
# settings
```



Remember:

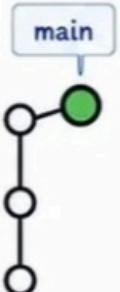
Use **--global** to apply settings for all repositories on your system.
Use **--local** for a specific repository.



Default Branch

Git's default branch name is changing from 'master' to 'main'.

```
$ git config --global init.defaultBranch main
# sets the default branch name
# new repositories will start with 'main'
```



6 Check Configuration Values



Check Username

```
$ git config user.name
Your Name
```



Check Email

```
$ git config user.email
youremail@example.com
```



Check Default Branch

```
$ git config init.defaultBranch
main
```



Check All Settings

```
$ git config --list
# shows all configuration
# values
```



Best Practice:

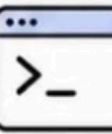
Always configure Git properly before starting any project. It saves time and avoids commit mistakes.





ESSENTIAL GIT COMMANDS

4



Your everyday Git toolkit - simple commands, powerful impact!

1 git init



Initializes a new Git repository in your project.

```
$ git init
```

```
Initialized empty  
Git repository in  
my-project/.git/
```

2 git clone



Clones an existing remote repository to your local machine.

```
$ git clone <url>
```

```
Cloning into 'repo'...  
remote: Enumerating  
objects: 120, done.  
Receiving objects: 100%
```

3 git status



Shows the current status of your working directory and staging area.

```
$ git status
```

```
On branch main  
Changes not staged:  
  modified: app.js  
Untracked files:  
  test.js
```

4 git add



Adds changes from working directory to the staging area.

```
$ git add <file>  
$ git add .
```

```
(no output)  
Moves changes  
to staging area.
```

5 git commit



Commits the staged changes to the local repository.

```
$ git commit -m  
"Your message"
```

```
[main a1b2c3d] Your message  
1 file changed, 5 insertions(+)  
create mode 100644 app.js
```

6 git log



Shows the commit history.

```
$ git log --oneline
```

```
a1b2c3d (HEAD -> main) Your message  
d4e5f6g Second commit  
h7i8j9k Initial commit
```

7 git diff



Shows differences between changes. Useful to review what you modified.

1) Working Directory vs Staging

```
$ git diff
```

```
- old line  
+ new line
```

2) Staged vs Last Commit

```
$ git diff --staged
```

```
- old line  
+ new line
```

3) Between Commits/Branches

```
$ git diff <a> <b>
```

```
- old line  
+ new line
```



QUICK RECAP

- Working Directory → You work here
- Staging Area → Prepare changes
- Local Repository → Save changes locally
- Remote Repository → Share with others



PRO TIPS

- ✓ Check status often using `git status`.
- ✓ Commit small, meaningful changes.
- ✓ Write clear commit messages.
- ✓ Pull before you push.

REMEMBER

- `add` → moves to staging
- `commit` → saves locally
- `push` → sends to remote



Small commits, clear messages, happy team! Keep your history clean and your code safe. 😊

Complete Git Workflow

Here is the complete end-to-end Git workflow you follow in any real project.



Follow this flow every time!

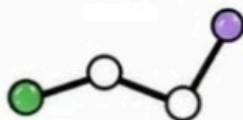
1 Clone



Copy a remote repository to your local machine.

```
$ git clone <url>
```

2 Branch



Create a new branch for your feature or fix.

```
$ git checkout -b feature/login
```

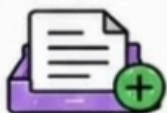
3 Code



Make changes in your files.

```
// Edit, Add or Update your code
```

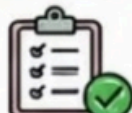
4 Add



Add your changes from working directory to staging area.

```
$ git add .
```

5 Commit



Commit your staged changes to local repository.

```
$ git commit -m "Your message"
```

6 Pull



Pull latest changes from remote repository.

```
$ git pull origin main
```

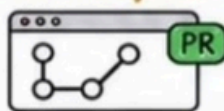
7 Push



Push your committed changes to remote.

```
$ git push origin feature/login
```

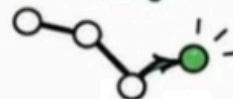
8 Pull Request



Create a Pull Request to review your changes.

```
// Open PR on GitHub, GitLab, Bitbucket etc.
```

9 Merge

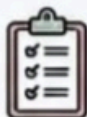


After review & approval, merge your PR.

```
// Changes are merged into main branch.
```

★ Key Points

- Always work on a **separate branch**.
- Pull latest changes before pushing.
- Write meaningful commit messages.
- Review before creating Pull Request.



💡 Tips

- Small commits, clear messages.
- Keep your branch up to date.
- Never push directly to main branch.
- Use .gitignore to avoid unnecessary files.



📖 Legend

- A, B, C = main commits
- D, E = feature commits (before rebase)
- D', E' = feature commits (after rebase)
- M = merge commit

Remember

Rebase rewrites history.
Use it carefully.

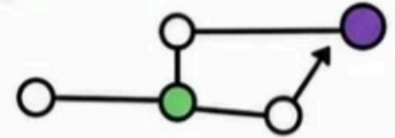
Merge preserves history.
Use it safely.



Git Branching

6

Work on new **features**, **fixes** or **experiments** without affecting the **main** code.



1 Create Branch

```
$ git branch <branch-name>
# Creates a new branch
```

Example:

```
$ git branch feature/login
```

2 Switch Branch

```
$ git switch <branch-name>
# Switch to a branch
```

Example:

```
$ git switch feature/login
```

3 Create & Switch

```
$ git switch -c <branch-name>
# Create and switch
```

Example:

```
$ git switch -c feature/login
```

4 Rename Branch -m → move

```
$ git branch -m <new-name>
# Rename current branch
```

Example:

```
$ git branch -m feature/auth
```

5 Delete Branch

```
$ git branch -d <branch-name>
# Delete merged branch
```

```
$ git branch -D <branch-name>
# Force delete (not merged)
```

```
$ git branch -D temp-branch
```



Tip

- Use `-d` to delete safely only if merged.
- Use `-D` to force delete unmerged branches.



6 Branch Naming Conventions

Type	Prefix	Purpose	Examples
Feature	feature/	New feature development	feature/login, feature/payment
Bug Fix	bugfix/	Bug fixes	bugfix/header-issue, bugfix/typo
Hotfix	hotfix/	Urgent fixes in production	hotfix/critical-login, hotfix/payment
Release	release/	Release preparation	release/v1.0.0, release/v2.1.0
Chore	chore/	Maintenance / dependencies / tasks	chore/update-deps, chore/docs

★ Best Practice

- Use lowercase letters.
- Use hyphen (-) not underscore (_).
- Keep it short, clear and meaningful.
- Follow team standards.



7 View Branches

```
***
$ git branch
# List local branches
```

```
***
$ git branch -r
# List remote branches
```

8 Branch Workflow Example



★ Remember

Branches are lightweight and extremely powerful. They make collaboration smooth and safe.



Branches keep your project **organized** and your code **safe**. Create, switch, collaborate and merge with **confidence**!





MERGE vs REBASE



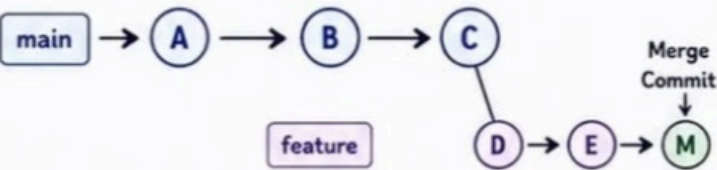
7

Both Merge and Rebase are used to integrate changes, but the history they create is **different**.

1

MERGE

Combines histories by creating a new **merge** commit.

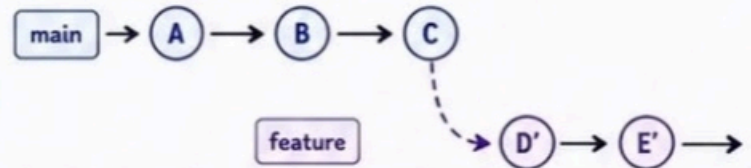


```
$ git merge feature
# merges feature into main
```

2

REBASE

Replays your commits on top of the latest commit.



```
$ git rebase main
# move feature commits to latest main
```

3

KEY DIFFERENCES

Aspect	MERGE	REBASE
History	Keeps complete history with merge commit.	Creates linear, clean history.
Commits	Original commits are preserved.	Creates new commits (different IDs).
Clarity	History can be messy with many merges.	History looks clean and easy to read.
Collaboration	Safe for shared branches.	Avoid rebase on shared branches.
Conflicts	Conflicts resolved once during merge.	Conflicts may appear multiple times.
Use Case	Feature integration, long running branches.	Updating feature branches before PR.

4

ADVANTAGES

MERGE

- ✓ Safe and easy for teams.
- ✓ Preserves the **complete** project history.
- ✓ Good for long feature branches.
- ✓ Ideal for release & integration.

REBASE

- ✓ Keeps history clean and linear.
- ✓ Easier to read and understand.
- ✓ Removes unnecessary merge commits.
- ✓ Great for feature branches.



TIP

Think of Merge as “**preserving history**” and Rebase as “**rewriting history**”.



5

WHEN TO USE?

USE MERGE WHEN:

- ✓ Working on team/shared branches
- ✓ Branch is public
- ✓ You want full history
- ✓ Long running features

USE REBASE WHEN:

- ✓ Working on your own feature branch
- ✓ Branch is not shared
- ✓ Before creating Pull Request
- ✓ You want clean history



6

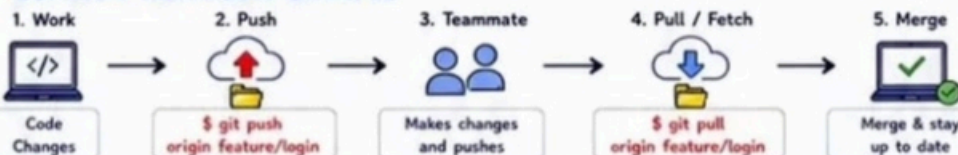
COMPANY PRACTICES

- ✓ Never rebase on main, develop or release branches.
- ✓ Always rebase your feature branch with latest main before PR.
- ✓ Use merge commit (or Squash) while merging PR.
- ✓ Follow team's Git workflow policy.



7

COMMON WORKFLOW EXAMPLE



LEGEND

A, B, C = main commits
D, E = feature commits (before rebase)
D', E' = feature commits (after rebase)
M = merge commit



REMEMBER

Rebase rewrites history.
Use it carefully.

Merge preserves history.
Use it safely.





Remote Repository

8

— Share, collaborate and keep your project safe in the cloud.

**1 Push**

Sends your local commits to the remote repository.

```
git push <remote> <branch>
# push local commits to remote
```

2 Pull

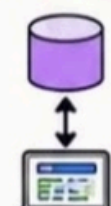
Fetches and merges changes from remote repository to local.

```
$ git pull <remote> <branch>
# fetch + merge
```

3 Fetch

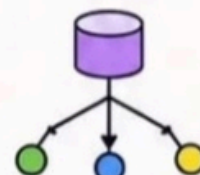
Downloads changes from remote but does NOT merge automatically.

```
$ git fetch <remote>
# update remote tracking branches
```

4 Origin

Default name of the remote repository you cloned from.

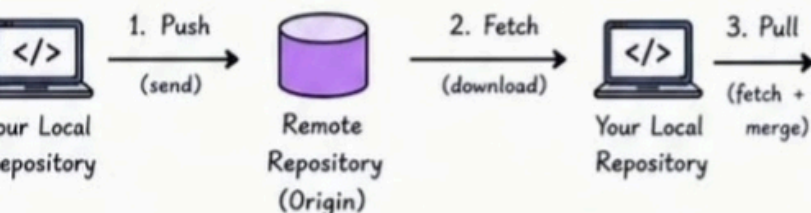
```
$ git remote -v
# view remote repositories
```

5 Upstream Branches

Remote branches that your local branches track and sync with.

```
$ git branch -vv
# show tracking information
```

How They Work Together



Check Remotes

```
$ git remote -v
origin https://github.com/user/repo.git (fetch)
origin https://github.com/user/repo.git (push)
```

'origin' is the default remote name.

Add / Remove / Rename Remote

Action	Command	Description
Add Remote	<code>\$ git remote add <name> <url></code>	Add a new remote repository
Remove Remote	<code>\$ git remote remove <name></code>	Remove an existing remote
Rename Remote	<code>\$ git remote rename <old> <new></code>	Rename a remote

Remote Tracking Branches (Upstream)



When you set an **upstream**, your local branch knows which remote branch to track.

Common Workflow Example



Tips

- Always pull before you push.
- Use meaningful branch names.
- Set upstream when creating a new branch.
- Keep your local repo in sync with remote.

Remember

- **Push** → upload your work to remote.
- **Pull** → get latest changes and merge.
- **Fetch** → download without merging.
- **Origin** → default remote name.



UNDO & RECOVERY



9

Git gives you powerful tools to undo mistakes and recover your project safely.

1 git restore

Discards changes in working directory or unstage a file.

```
$ git restore <file>
$ git restore --staged <file>
# discard changes in working directory
# unstage a file
```

? When to use?

When you want to discard local changes that are not committed.

2 git reset

Moves HEAD to a specific commit and updates staging & working directory based on mode.

```
$ git reset --soft <commit> # keep staged
$ git reset --mixed <commit> # keep unstaged
$ git reset --hard <commit> # discard all
```

? When to use?

When you need to undo commits. Use with caution!



3 git revert

Creates a new commit that undoes the changes of a previous commit.

```
$ git revert <commit>
# safe undo (does not change history)
```

? When to use?

When you want to undo commits on shared/public branches.



4 git checkout

Switch branches or restore files from a specific commit.



```
$ git checkout <branch>
$ git checkout <commit> -- <file>
# switch branch
# restore file from commit
```

? When to use?

When switching branches or restoring a file from a specific commit.



5 git reflog

Shows a log of all HEAD movements. Helps to recover lost commits.

```
$ git reflog # show reflog
```

Example Output:

```
a1b2c3d HEAD@{0}: reset: moving to a1b2c3d
d4e5f6g HEAD@{1}: commit: add login test
h7i8j9k HEAD@{2}: commit: initial commit
```

? When to use?

When you lost commits due to reset or want to go back to old states.



Quick Comparison



Command	What it does	Affects History?
restore	Discard changes	No
reset	Move HEAD	Yes (hard) No (soft/mixed)
revert	Undo by new commit	No
checkout	Switch / Restore	No
reflog	Show history of moves	No



When to Use Each?

Discard Changes



- Use **git restore**
- When you made changes but didn't commit.

Undo Local Commits



- Use **git reset**
- When commits are local and not pushed.

Undo Shared Commits



- Use **git revert**
- When commits are already pushed.

Switch / Restore Files



- Use **git checkout**
- When switching branches or restoring files.

Recover Lost Commits



- Use **git reflog**
- When commits are lost due to reset or other actions.



Best Practices

- ✓ Avoid using **git reset --hard** on shared branches.
- ✓ Always double-check before discarding changes.
- ✓ Use generically: prefer **revert** over **reset** on public branches.
- ✓ **reflog** is your best friend in recovery!



Tip

Before any major undo, create a backup branch.

```
$ git branch backup-$(date +%Y%m%d)
```

Better safe than sorry!



Legend

- A, B, C = main commits
- D, E = feature commits (before rebase)
- D', E' = feature commits (after rebase)
- M = merge commit



Remember

Rebase rewrites history.
Use it carefully.

Merge preserves history.
Use it safely.